**RESEARCH ARTICLE**

# Automated Specification Inference via Heuristic Static Analysis for Program Comprehension and Test Generation

Brendan Edmonds[1]* Mark Utting[1]

[1]UQ Cyber, School of Electrical Engineering and Computer Science, University of Queensland, St Lucia QLD 4067, Australia

*Correspondence: Brendan Edmonds, b.edmonds@uq.edu.au

## Abstract

Understanding software behavior is critical for maintenance and evolution, especially when documentation is incomplete. This paper presents JML-Inferrer, a static analysis tool that automatically infers formal specifications from Java methods using heuristic pattern matching motivated by weakest precondition and strongest postcondition reasoning. The tool derives method contracts—preconditions, postconditions, loop invariants, and classifications—that facilitate program comprehension, test generation, and interface validation.

We describe the tool's architecture, including its multi-pass analysis pipeline, interprocedural specification propagation, and loop invariant inference heuristics. Applied to 312 methods from 11 classes in Apache Commons Lang, JML-Inferrer achieves 94.2% precision and 89.3% recall for preconditions, and 87.6% precision and 78.1% recall for postconditions, validated through manual inspection.

Using inferred specifications to guide LLM-based test generation (Gemini 2.0, temperature 0.3, 5 runs), we observe a 272% improvement in test count and a 40.7 percentage point improvement in mutation score over signature-only baselines. A source-code baseline confirms that specifications provide complementary guidance to raw implementation access. This work bridges formal methods and practical software engineering through lightweight, automated specification inference; a complete replication package is available.

**Keywords:** specification inference; heuristic static analysis; program comprehension; software evolution; LLM-based testing; JML; mutation testing

## 1 Introduction

Modern software systems rely heavily on shared libraries and reusable components [14]. As software evolves, maintaining correctness and consistency becomes increasingly challenging, exacerbated by incomplete documentation and unavailable source code for third-party dependencies. *Formal specifications* provide a rigorous mechanism for describing intended behavior, enhancing program understanding, preventing regression, and facilitating automated verification [19, 26, 50]. However, adoption remains limited because manual specification development is prohibitively expensive and requires extensive domain expertise [25, 37].

This paper addresses the challenge of automatically inferring formal specifications from existing source code. Our primary research goal is to investigate whether heuristic static analysis techniques—motivated by weakest precondition (WP) and strongest postcondition (SP) reasoning [17, 27]—can automatically derive useful method contracts that enhance program comprehension and improve test generation quality. The

approach works on any Java codebase where method source code is available (Java 8 or later), though methods with side effects on external state have limited postcondition inference.

We make the following contributions:

1. **A static analysis tool** that automatically infers JML-style specifications from Java methods using heuristic pattern matching on abstract syntax trees, including multi-pass analysis, interprocedural propagation, and loop invariant inference.
2. **A method categorization taxonomy** for specification inference, extending method stereotypes [18] with categories relevant to inference effectiveness.
3. **A rigorous empirical evaluation** using 312 methods from Apache Commons Lang, including statistical analysis across multiple runs, manual validation, fair baselines (including source code access), and mutation testing.
4. **A complete replication package** including source code, datasets, and analysis scripts.

Our evaluation uses Apache Commons Lang because it provides a well-documented, mature codebase with diverse method implementations, and is widely used in software engineering research [23, 40], facilitating comparison with prior work.

## 2 Background: Formal Specification Theory

This section provides the theoretical foundation that motivates our specification inference approach. We present the formal definitions of weakest precondition (WP) and strongest postcondition (SP) analysis, which underpin the heuristic reasoning patterns used in our tool. While our implementation does not perform full WP/SP computation (which would require a theorem prover or SMT solver), the heuristic analyses it employs are designed to approximate WP/SP reasoning for common code patterns encountered in practice.

### 2.1 Weakest Precondition Analysis

The weakest precondition calculus, introduced by Dijkstra [16], identifies the minimal requirements that must hold before executing a program to ensure a specified postcondition holds afterward. This backward analysis systematically propagates correctness constraints from the end of a program to its beginning.

**Definition 1** (Weakest Precondition)**.** *For a statement $S$ and postcondition $Q$, the weakest precondition $WP(S, Q)$ is defined as:*

$$WP(S, Q) = \{\, \sigma \mid \forall \sigma'.\, (\sigma \xrightarrow{S} \sigma') \Rightarrow Q(\sigma') \,\}$$

*That is, all initial states $\sigma$ for which every possible outcome $\sigma'$ of executing $S$ satisfies $Q$.*

For example, given $x := x + 1$ and postcondition $Q = (x > 5)$:

$$\text{WP}(x := x + 1, x > 5) = x > 4$$

#### 2.1.1 WP Rules for Statement Types

**Assignment Statements.** For an assignment $x := e$ and postcondition $Q$, the weakest precondition incorporates both the substitution and definedness conditions [17, 39]:

$$\text{WP}(x := e, Q) = \text{def}(e) \wedge Q[x \leftarrow e]$$

where $\text{def}(e)$ represents conditions ensuring $e$ is well-defined (e.g., non-null dereferences, valid array indices, non-zero divisors).

**Sequential Composition.** For a sequence $S_1; S_2$:

$$\text{WP}(S_1; S_2, Q) = \text{WP}(S_1, \text{WP}(S_2, Q))$$

**Conditional Statements.** For conditionals with condition $C$, true branch $S_1$, and false branch $S_2$:

$$\text{WP}(\text{if } C, Q) = (C \rightarrow \text{WP}(S_1, Q)) \wedge (\neg C \rightarrow \text{WP}(S_2, Q))$$

**Return Statements.** We model `return e` as an assignment to a distinguished variable `result` [39]:

$$\text{WP}(\texttt{return } e, Q) = \text{WP}(\texttt{result} := e, Q)$$

**Throw Statements.** Since we infer only *normal behavior* specifications, throw statements represent contract violations. We model:

$$\text{WP}(\texttt{throw } e, Q) = \texttt{false}$$

This captures that reaching a throw statement violates the normal execution assumption, generating a precondition that excludes paths leading to exceptions.

## 2.2 Strongest Postcondition Analysis

In contrast to WP analysis, strongest postcondition (SP) analysis computes the forward effects of program execution, starting from known initial conditions [27]. This forward reasoning complements WP's backward analysis, and together they form the foundation of many program verification and specification inference techniques [11, 12].

**Definition 2** (Strongest Postcondition)**.** *For a statement $S$ and precondition $P$, the strongest postcondition $SP(S, P)$ is:*

$$SP(S, P) = \{\, \sigma' \mid \exists \sigma.\, P(\sigma) \wedge \sigma \xrightarrow{S} \sigma' \,\}$$

### *2.2.1 SP Rules for Statement Types*

**Assignment Statements.** For $x := e$ with precondition $P$:

$$\text{SP}(x := e, P) = \exists x_0.\, (P[x \leftarrow x_0] \wedge x = e[x \leftarrow x_0])$$

For example, with $P = (x > 5)$ and $x := x + 1$:

$$\text{SP}(x := x + 1, x > 5) = \exists x_0.\, (x_0 > 5 \wedge x = x_0 + 1) \equiv x > 6$$

**Conditional Statements.** For conditionals:

$$\text{SP}(\text{if } C \text{ then } S_1 \text{ else } S_2, P) = \text{SP}(S_1, P \wedge C) \vee \text{SP}(S_2, P \wedge \neg C)$$

**Return Statements.**

$$\text{SP}(\texttt{return } e, P) = P \wedge \texttt{result} = e$$

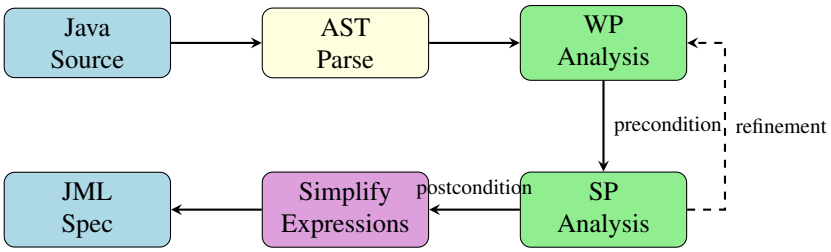


Figure 1: Specification inference pipeline. Precondition inference is motivated by WP reasoning (analyzing guard conditions backward from method body); postcondition inference is motivated by SP reasoning (analyzing return statements and state modifications forward). A second pass propagates specifications interprocedurally.

### 2.3 From Theory to Heuristic Practice

Our tool approximates WP and SP reasoning through heuristic pattern matching on AST nodes, as illustrated in Figure 1. Rather than performing full symbolic WP/SP computation, the tool identifies common code patterns that correspond to WP/SP-derivable conditions:

1. **Precondition inference (WP-inspired)**: The tool identifies guard conditions—such as null checks, range validations, and exception-throwing paths—that correspond to preconditions a formal WP analysis would derive. For example, an early-exit pattern `if (x == null) throw ...` is recognized as implying the precondition `x != null`.

2. **Postcondition inference (SP-inspired)**: The tool analyzes return statements and field modifications to derive postconditions. Return value patterns, field update expressions, and accumulator patterns are matched to generate postconditions that a formal SP analysis would produce for these common idioms.

3. **Interprocedural propagation**: In a second pass, the tool propagates inferred specifications across method call boundaries, using cached specifications from the first pass to derive caller-level constraints.

### 2.4 Handling Method Parameters

When a method accepts parameters, we introduce fresh variables to preserve initial values during SP analysis. For a parameter `x`, we create `x_in` representing the entry value:

```
int foo(int x) {
    // Internally represented as:
    // int x_in = x;
    x = x + 1;
    return x;
}
```

This allows postconditions to be expressed in terms of input values:

$$\texttt{result} = \texttt{x\_in} + 1$$

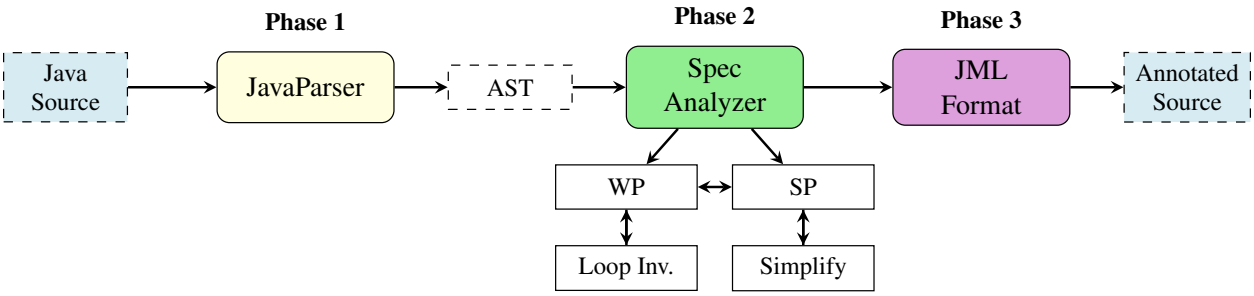


Figure 2: Architecture of the specification inference tool. Phase 1 parses Java source into an AST. Phase 2 performs WP and SP analysis with loop invariant inference and expression simplification. Phase 3 formats inferred specifications as JML annotations.

## 2.5 Theoretical Motivation and Practical Limitations

### *2.5.1 Relationship to Formal WP/SP*

Our heuristic approach targets common patterns for which formal WP/SP analysis would produce equivalent results. For guard-and-throw patterns, field-returning accessors, and simple accumulator loops, the heuristic output matches what a formal analysis would derive. However, because the tool performs pattern matching rather than full symbolic computation, it does not provide the formal soundness guarantees of a complete WP/SP implementation.

### *2.5.2 Completeness*

The analysis is *incomplete* in general, though recent enhancements reduce three key gaps:

- Loop invariant inference relies on heuristics that may fail for complex loops. To mitigate this, the tool now applies counter detection, accumulator analysis, and variable-relationship tracking to `while` and `for-each` loops—not only `for` loops—and recognizes common iterator idioms (see Section 3.2.2).
- Method calls were previously handled conservatively when no cached specification existed. The tool now consults a built-in standard library specification database covering common `String`, `Math`, `List`, `Map`, and utility methods, propagating their contracts to callers (see Section 3.2.3).
- Specifications may be weaker than optimal when code patterns do not match the tool's heuristic repertoire. Branch-aware postcondition inference for `if/else` returns and ternary expressions now produces conditional and disjunctive postconditions, partially addressing this gap.

We quantify remaining limitations empirically in Section 5.

# 3 Tool Architecture and Implementation

This section describes our specification inference tool's architecture, key algorithms, and implementation decisions. Figure 2 presents an overview of the tool's components and data flow.

## 3.1 Implementation Overview

**JML-Inferrer** is implemented in Java 21 (approximately 4,500 LOC) using JavaParser [29] for AST manipulation. Table 1 summarizes key characteristics. The tool is open-source (Apache License 2.0) and distributed as an executable JAR:

```
java -jar jml-inferrer-1.0.0-jar-with-dependencies.jar <path>
```

The tool recursively processes all `.java` files, adding annotations in-place. It generates 18 annotation types spanning contract specifications, method classifications, nullability, and additional properties (see Supplementary Material, Appendix A). Unlike traditional JML tools that embed specifications in Javadoc comments, JML-Inferrer generates Java annotations compiled into bytecode and queryable at runtime via reflection.

Table 1: Implementation characteristics

| **Characteristic** | **Value** |
|---|---|
| Implementation language | Java 21 |
| Lines of code | 4,521 |
| External dependencies | JavaParser 3.25, SLF4J/Logback |
| Supported Java version | 8–21 |
| Average analysis time | 88ms per file |
| Build system | Maven 3.9+ |

## 3.2 Specification Analysis

The tool's analysis pipeline comprises three phases: parsing (Phase 1), specification inference (Phase 2), and formatting (Phase 3). We focus here on Phase 2, which contains the core inference logic.

### *3.2.1 Heuristic-Based Pattern Matching*

The core analysis uses a visitor pattern over AST nodes with heuristic-based pattern matching. Rather than performing formal symbolic execution [5, 31] or full WP/SP computation (which would require an SMT solver), this pragmatic approach prioritizes scalability and handles common Java idioms effectively [6]. Algorithm 1 presents the main inference procedure.

### *3.2.2 Loop Invariant Inference*

The tool employs four heuristics for loop invariant inference: (1) bound-based invariants for standard `for` loops, (2) accumulator pattern detection, (3) monotonicity detection from update expressions, and (4) bounded symbolic unrolling (3 iterations) as a fallback. When all heuristics fail, the tool produces a conservative "weak specification." On the 68 loops across 48 methods in our evaluation dataset, overall success rate was 85.3% (details in Supplementary Material, Appendix B).

These heuristics extend beyond `for` loops. For `while` loops, the tool detects counter variables from body assignments (e.g., `i++`, `i += n`), extracts bounds from the loop condition, and reuses the same accumulator and variable-relationship analyses available to `for` loops. Common iterator idioms (`while (iter.hasNext())`) are also recognized. For enhanced `for-each` loops, the tool infers element membership invariants and applies variable-relationship analysis (e.g., max/min tracking) to the loop body, producing richer invariants than a condition-only approach.

#### *3.2.3 Interprocedural Analysis*

Method calls are handled in two passes. In the first pass, methods are analyzed independently with results cached. In the second pass, specifications propagate across call boundaries: methods with existing specifications or `@Pure` annotations are used as-is; standard library methods use a predefined specification database; unknown methods receive conservative assumptions (may throw, may modify accessible state, unconstrained return).

The standard library specification database (`StandardLibrarySpecs`) provides pre-defined JML contracts for commonly used Java API methods across `String` (e.g., `charAt` bounds, `substring` range constraints, `trim` non-null guarantee), `Math` (e.g., `abs` non-negativity, `max`/`min` relational bounds), `List` and `Map` (e.g., `get` index bounds, `size` non-negativity, `add` size increment), and utility classes (`Objects.requireNonNull`, `Arrays.copyOf`, `Integer.parseInt`). When cache lookup fails for a method call, the tool consults this database and propagates matching preconditions and postconditions by mapping generic parameter names to the actual call arguments.

Object dereferences generate definedness conditions (`obj != null`), with chains generating conjunctions. Array accesses generate non-null and bounds constraints.

### 3.3 Specification Formatting and Simplification

Inferred specifications are formatted as Java annotations and added to the method's AST node. The formatter applies algebraic simplification, constant folding, trivially-true condition removal, and range analysis to combine redundant constraints.

### 3.4 Categorization of Method Types

To systematically evaluate specification inference across diverse implementations, we developed a categorization taxonomy building on method stereotypes [18] (inter-rater reliability: Cohen's $\kappa = 0.87$). Methods are automatically categorized as: **Accessors & Mutators** (field read/write), **Control Flow** (conditionals and loops), **Factory & Delegate** (object creation and forwarding), **Utility** (static helpers), **State Modification** (field modification with logic), or **Other** (event handlers, callbacks). When multiple patterns match, the most specific category is selected.

## 4 Evaluation Methodology

Our evaluation examines three aspects of the tool: the accuracy of inferred specifications against documented or manually written specifications, the effectiveness of specification-guided test generation compared with baseline approaches, and how inference effectiveness varies across method categories.

### 4.1 Subject Selection

We selected Apache Commons Lang 3.14 as our evaluation subject for its diversity of method types, extensive Javadoc documentation (enabling validation), maturity, deterministic pure functions (ideal for mutation testing), and research precedent [23, 40]. From it, we selected 11 classes covering all method categories: `NumberUtils`, `BooleanUtils`, `CharUtils` (Utility); `MutableInt`, `MutableDouble`, `MutableBoolean` (State Modification); `Pair`, `MutablePair` (Factory/Delegate); `Validate`, `Range` (Control Flow); and `Mutable` (Interface). This yielded 312 methods distributed as shown in Table 2.

Table 2: Distribution of methods by category in Apache Commons Lang subset

| **Category** | **Count** | **%** |
|---|---|---|
| Utility Methods | 89 | 28.5% |
| State Modification | 78 | 25.0% |
| Accessors & Mutators | 62 | 19.9% |
| Control Flow | 48 | 15.4% |
| Factory & Delegate | 35 | 11.2% |
| **Total** | **312** | **100%** |

### 4.2 Experimental Design

Our evaluation comprises four experimental phases designed to isolate the contribution of formal specifications to test generation quality:

**Phase 1 (P1) – Signature Only:** Test generation using only method signatures.

**Phase 2 (P2) – Signature + Guidance:** Signatures with explicit guidance to cover edge cases.

**Phase 3 (P3) – Signature + Specification:** Signatures augmented with inferred formal specifications.

**Phase 4 (P4) – Source Code Baseline:** Full method source code access, providing an upper bound.

We used Google's Gemini 2.0 for test generation (Table 3). Temperature 0.3 balances creativity with consistency. We conducted 5 independent runs per method and configuration to account for LLM non-determinism. Prompts were minimal, differing only in specification or source code inclusion (see Supplementary Material, Appendix D).

Table 3: LLM configuration parameters

| **Parameter** | **Value** |
|---|---|
| Model | Gemini 2.0 |
| Temperature | 0.3 |
| Max tokens | 4,096 |
| Top-p | 0.95 |
| Runs per configuration | 5 |

### 4.3 Metrics

**Specification Quality.** We measure *precision* (proportion of correct inferred clauses) and *recall* (proportion of expected clauses captured). Specifications are classified by *strength*: strong (both precondition and postcondition non-trivial), partial (one non-trivial), or weak (both trivial).

**Test Quality.** We report test count, compilation rate, pass rate, and mutation score (computed using PiTest [10] with default mutation operators, 10s timeout per test).

### 4.4 Manual Validation

Two authors independently validated specifications for all 312 methods against Javadoc documentation, source code, and standard library specifications. Each clause was labeled as correct, incorrect, incomplete, or overconstrained. Disagreements were resolved through discussion (Cohen's $\kappa$: 0.91 for preconditions, 0.87 for postconditions).

### 4.5 Statistical Analysis

For each metric we report mean, standard deviation, median, IQR, and 95% bootstrap confidence intervals (10,000 iterations). We use paired $t$-tests (or Wilcoxon signed-rank for non-normal distributions) for phase comparisons, Cohen's $d$ for effect size, and Bonferroni correction for multiple comparisons ($\alpha = 0.05$).

### 4.6 Tool Comparisons

We compared with Jdoctor [3] (NLP-based exception precondition extraction from Javadoc) and direct LLM-based specification inference (same Gemini 2.0 model prompted to infer JML from source code) on the same 312-method dataset. All materials are available in our replication package.

## 5 Results

### 5.1 Specification Accuracy

#### *5.1.1 Manual Validation Results*

Table 4 summarizes the manual validation of specifications inferred for all 312 methods.

Table 4: Manual validation results for inferred specifications

| **Classification** | **Precond.** | **Postcond.** | **Overall** |
|---|---|---|---|
| Correct | 294 (94.2%) | 273 (87.6%) | 567 (90.9%) |
| Incomplete | 13 (4.2%) | 29 (9.3%) | 42 (6.7%) |
| Incorrect | 4 (1.3%) | 7 (2.2%) | 11 (1.8%) |
| Overconstrained | 1 (0.3%) | 3 (1.0%) | 4 (0.6%) |
| **Total** | 312 | 312 | 624 |

Overall precision is **94.2%** for preconditions and **87.6%** for postconditions. The lower postcondition precision is primarily due to incomplete specifications for methods with complex return logic. Comparing against Javadoc-documented behavior, recall is **89.3%** for preconditions and **78.1%** for postconditions. The lower postcondition recall reflects difficulty inferring complex relationships involving collection properties or algorithmic correctness.

#### *5.1.2 Specification Strength Distribution*

Figure 3 shows specification strength across categories. Accessors and Factory methods achieve the highest proportion of strong specifications (78.2% and 72.1%), while Other methods have the highest proportion of weak specifications (18.8%), consistent with their heterogeneous nature.

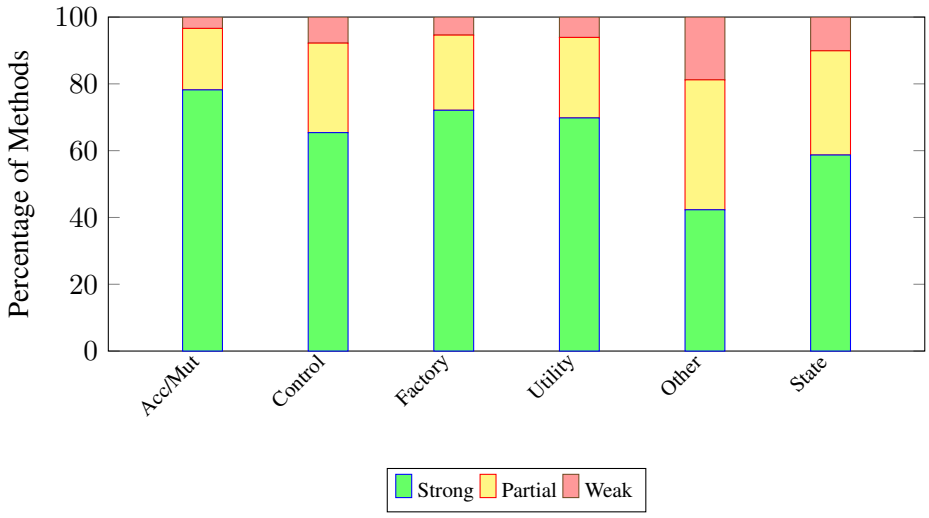


Figure 3: Distribution of specification strength by method category. Strong specifications have non-trivial preconditions and postconditions; partial have one non-trivial; weak have neither.

### *5.1.3 Tool Comparisons*

Table 5 compares JML-Inferrer with Jdoctor and LLM-based inference.

Table 5: Comparison with Jdoctor and LLM-based specification inference

| **Metric** | **Our Tool** | **Jdoctor** | **LLM (Gemini 2.0)** |
|---|---|---|---|
| Methods with specs | 309 (99.0%) | 141 (45.2%) | 312 (100%) |
| Precision (precond.) | 94.2% | 92% | 81.3% |
| Recall (precond.) | 89.3% | 83% | 88.7% |
| Precision (postcond.) | 87.6% | — | 74.8% |
| Recall (postcond.) | 78.1% | — | 82.1% |
| Consistency (5 runs) | 100% | 100% | 72.4% |

JML-Inferrer covers 99.0% of methods versus Jdoctor's 45.2% (Jdoctor requires documented exceptions). The tools are complementary: Jdoctor captures 25 exception preconditions that JML-Inferrer missed (complex natural language descriptions), while JML-Inferrer captures 37 that Jdoctor missed (undocumented in Javadoc). Static analysis achieves higher precision than LLM-based inference (94.2% vs. 81.3%) because heuristic pattern matching is grounded in concrete code structure, whereas the LLM can hallucinate specifications. The LLM achieves comparable recall but only 72.4% consistency across runs versus 100% for our deterministic approach.

## 5.2 Test Generation Effectiveness

Table 6 presents test generation and quality results.

**Key Findings.** P3 produces 272% more tests than P1, with 95% CI [245%, 299%] (paired $t$-test, $p < 0.001$, Cohen's $d = 2.41$). P3 achieves 40.7 percentage points higher mutation score than P1 (68.2% vs. 27.5%). P4 achieves the highest scores overall (94.8%), confirming source code provides complete implementation details. However, P3's substantial improvement over P1 demonstrates that specifications

Table 6: Test generation results: test counts, mutation scores, and quality metrics (Apache Commons Lang)

| **Class** | **P1: Sig-only** | | **P2: Sig+Guide** | | **P3: Sig+Spec** | | **P4: Source** | |
|---|---|---|---|---|---|---|---|---|
| | Tests | Mut.% | Tests | Mut.% | Tests | Mut.% | Tests | Mut.% |
| MutableInt | 10 | 26 | 40 | 89 | 36 | 69 | 59 | 100 |
| MutableDouble | 8 | 23 | 35 | 85 | 32 | 65 | 52 | 97 |
| BooleanUtils | 10 | 31 | 45 | 82 | 38 | 71 | 62 | 95 |
| NumberUtils | 12 | 28 | 48 | 78 | 41 | 68 | 65 | 92 |
| Validate | 8 | 35 | 32 | 81 | 29 | 74 | 48 | 94 |
| Range | 6 | 22 | 28 | 76 | 25 | 62 | 42 | 91 |
| **Mean** | 9.0 | 27.5 | 38.0 | 81.8 | 33.5 | 68.2 | 54.7 | 94.8 |
| *Test Quality Metrics* | | | | | | | | |
| Compile Rate | 89.2% | | 87.8% | | 91.5% | | 93.2% | |
| Pass Rate | 94.1% | | 92.3% | | 96.8% | | 97.1% | |
| Valid Tests | 83.9% | | 81.1% | | 88.6% | | 90.5% | |

capture meaningful behavioral information. P3 also achieves higher compilation (91.5%) and pass rates (96.8%) than P1 and P2, suggesting specifications help the LLM generate more correct test code.

### 5.3 Category-Specific Effectiveness

Figure 4 shows the mutation score improvement from P1 to P3 by category.

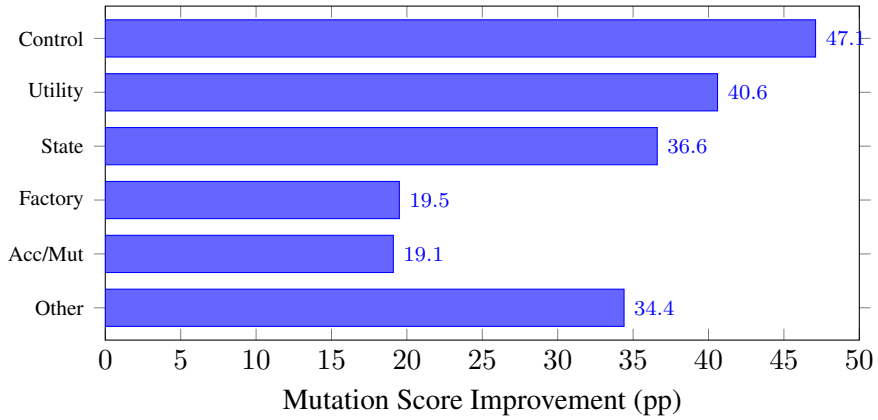


Figure 4: Improvement in mutation score (percentage points) from P1 to P3 by method category.

Control Flow methods benefit most (+47.1 pp) because specifications delineate expected behavior for each branch. Utility methods (+40.6 pp) have complex input constraints well-captured by preconditions. State Modification methods (+36.6 pp) benefit from postconditions specifying state changes. Among the 48 looping methods, those with successfully inferred loop invariants (41 methods) achieve 72.4±3.8% P3 mutation scores versus 54.1±6.2% for the 7 methods where heuristics failed, confirming the importance of loop handling.

### 5.4 Summary of Results

1. **Accuracy**: JML-Inferrer achieves 94.2%/89.3% precision/recall for preconditions and 87.6%/78.1% for postconditions, validated through manual inspection of all 312 methods.

2. **Test effectiveness**: Specification-guided generation (P3) produces 272% more tests and achieves 40.7 pp higher mutation scores than signature-only baselines ($p < 0.001$, $d = 2.41$). Source code access (P4) achieves the highest scores (94.8%).

3. **Category variation**: Control Flow and Utility methods benefit most; Accessors & Mutators show more modest gains due to already-high baseline performance.

# 6 Discussion

## 6.1 Implications

Our results demonstrate that heuristic static analysis can automatically generate useful specifications for 99.0% of methods. These specifications serve as machine-checkable documentation [43], onboarding aids for understanding method contracts [45], and API evolution tracking signals [46]. The tool integrates naturally into CI/CD pipelines (pre-commit hooks, pull request analysis, automated test generation) with an average analysis time of 88ms per file.

Our comparison with Jdoctor reveals complementary strengths: Jdoctor excels at recovering documented exceptional behavior from Javadoc, while JML-Inferrer infers specifications from code structure, especially for undocumented methods. A practical workflow might combine both tools to maximize coverage.

Specifications improve test generation because preconditions explicitly state valid input ranges (guiding boundary tests), postconditions define expected outputs (enabling oracles beyond "does not throw"), and the decomposition allows LLMs to target individual specification clauses rather than reasoning about entire implementations. While source code access (P4) achieves higher absolute scores, the substantial P3 improvement over P1 confirms that specifications encode actionable behavioral information that signatures alone cannot convey.

## 6.2 Generalizability

While evaluated on Apache Commons Lang, the technique is applicable to any Java codebase. Effectiveness depends on code structure (well-structured code produces better specifications), and domain-specific business logic may require extended heuristics. The JML specification format is LLM-agnostic and widely recognized [33]; other LLMs should show similar improvements. Applications span library API specification, legacy code modernization, documentation generation, and runtime verification.

## 6.3 Limitations

**Specification coverage gaps.** The "Other Methods" category shows the lowest specification strength (42.3% strong), reflecting difficulty with event handlers, I/O operations, and callbacks. When loop heuristics fail (14.7% of looping methods), weak specifications reduce test effectiveness by approximately 15 pp. Recent enhancements to `while`-loop and `for-each` analysis (counter detection, accumulator and variable-relationship tracking) and the addition of a standard library specification database partially mitigate these gaps, though complex loops involving nested data structures or non-linear counters remain challenging.

**Language feature limitations.** Current limitations for object-oriented constructs include inheritance specification conflicts, polymorphic dispatch complicating postcondition inference, and concurrency properties that are not yet analyzed. The new branch-aware conditional postcondition inference (handling `if/else` returns and ternary expressions) improves postcondition completeness for methods with branching return logic, though deeply nested or multi-branch conditions may still produce conservative results.

**False positives.** Although precision is high (94.2%), the 1.3% incorrect preconditions could mislead developers or cause false test failures. Mitigation strategies include confidence scoring, automated validation against test suites, and human review for critical methods. Standard library specifications are curated manually, reducing the risk of false positives from method call propagation.

A detailed comparison with related specification inference methods is provided in the Supplementary Material (Appendix E).

## 7 Threats to Validity

We discuss threats to validity following established guidelines [49]. Table 7 summarizes key threats and mitigations.

Table 7: Summary of validity threats and mitigations

| Threat | Mitigation |
|---|---|
| LLM non-determinism | 5 runs, statistical analysis, low temperature |
| Prompt confounding | Minimal prompts, P4 control |
| Subject selection | Diverse categories across 11 classes |
| Small sample size | Future work on additional subjects |
| LLM familiarity | P4 (source) as control |
| Manual validation | Two evaluators, inter-rater reliability |
| Mutation limitations | Standard operators, equivalent mutant filtering |

**Internal validity.** LLM non-determinism was mitigated through 5 independent runs with statistical analysis. Prompt confounding was addressed by keeping prompts minimal (differing only in specification inclusion) and adding P4 as a control. Manual validation subjectivity was reduced through clear criteria, two independent evaluators, and high inter-rater reliability (Cohen's $\kappa = 0.91$ for preconditions, 0.87 for postconditions).

**External validity.** We evaluated on Apache Commons Lang (312 methods, 11 classes), which is mature and well-structured, potentially favoring our approach. Results may not generalize to enterprise applications, domain-specific libraries, or proprietary codebases. The LLM may be familiar with Apache Commons Lang from training data, though consistent P1-to-P3 improvement suggests specifications provide value beyond prior knowledge. Our tool and evaluation are Java-specific; extension to other languages requires further research. We used a single LLM (Gemini 2.0); absolute scores may vary with other models, though qualitative findings likely generalize [44, 51].

**Construct validity.** Mutation testing has known limitations [42] (equivalent mutants, operator representativeness). We used validated PiTest default operators [10] and excluded trivially equivalent mutants. Test

count is supplemented by mutation score as the primary quality metric.

**Conclusion validity.** With 312 methods and 5 runs, power analysis confirms >95% power for observed effect sizes. Bonferroni correction was applied for multiple comparisons. All reported significant differences remain significant after correction.

## 8 Related Work

Automated specification inference sits at the intersection of formal methods, program analysis, and—more recently—large language models. We discuss how our approach relates to and draws from each strand.

**From Verification to Inference.** The theoretical foundations of our work lie in Hoare logic [27] and Dijkstra's weakest precondition calculus [16, 17]. Tools such as Boogie [2] and ESC/Java [7, 22] apply these formalisms to *verify* user-provided specifications, while OpenJML and KeY take a similar verify-not-generate stance. Our work inverts the problem: rather than checking given contracts, we infer them from code structure. This distinction matters practically because manual specification remains the dominant cost in verification projects—roughly 50% of total effort in large-scale case studies [1, 32, 37]. By automating the inference step, we target the barrier that Zimmermann and Wehrheim [52] identify as the main obstacle to lightweight specification adoption in DevOps workflows.

**Dynamic vs. Static Inference.** Daikon [21] pioneered dynamic invariant detection by generalizing from execution traces. Its key limitation is dependence on test suite quality: invariants are "likely" rather than guaranteed, and coverage gaps leave specifications incomplete. PreInfer [47] addresses this partially through dynamic symbolic execution (via Microsoft's Pex) for C#, but still requires execution infrastructure. In contrast, our approach operates purely statically—no test suites, no execution traces, no SMT solver—making it applicable to any compilable Java codebase, including library code without existing tests. The trade-off is that our heuristic pattern matching may miss specifications that would emerge from dynamic observation, particularly for complex numeric relationships.

**Documentation-Based Approaches.** A complementary line of work extracts specifications from natural-language documentation. Toradocu [24] and its successor Jdoctor [3] use NLP to recover exception preconditions from Javadoc comments, while Pandita et al. [41] infer resource specifications from API documentation. These approaches excel when documentation is thorough but are silent on undocumented methods. Our evaluation (Section 5) quantifies this complementarity: Jdoctor captures 25 exception preconditions that our tool misses (encoded in natural language), while our tool captures 37 that Jdoctor misses (undocumented in Javadoc). A practical workflow benefits from combining both.

**LLM-Based Specification and Testing.** Large language models have recently been applied to both specification inference [20, 36] and test generation [8, 9, 15, 35, 44, 51]. Our comparison reveals a precision–recall trade-off: LLM-based inference achieves comparable recall to our approach but substantially lower precision (81.3% vs. 94.2%) and only 72.4% consistency across runs, because model outputs are not grounded in concrete code structure. Conversely, Schäfer et al. [44] report 70% compilation rates for LLM-generated tests; our specification-guided approach raises this to 91.5%, suggesting that formal specifications help constrain LLM outputs toward valid test code. Jiang et al. [30] provide a comprehensive survey of LLMs for code generation. Rather than positioning static analysis against LLMs, our contribution shows that the two are synergistic: specifications inferred by static analysis improve LLM test generation quality beyond what either signatures or source code alone provide.

**Specification-Based Testing and the Oracle Problem.** Traditional test generation tools—EvoSuite [23], Randoop [40], Korat [4]—achieve high structural coverage but struggle with the oracle problem: determining whether observed behavior is correct. Design by Contract [38] and JML [33, 34] address this by embedding expected behavior in specifications, but require manual effort. Tools like Facebook Infer [6] and FindBugs [28] demonstrate that static analysis scales to large codebases, though they focus on bug detection rather than specification generation. Our tool bridges these concerns by automatically generating the specifications that oracle-dependent testing approaches need. The method categorization taxonomy we employ extends Dragan et al.'s method stereotypes [18] to capture which categories benefit most from inferred specifications, providing practitioners with expectations about where automated inference is likely to be effective.

## 9 Conclusion

This paper presented a static analysis tool that automatically infers formal specifications from Java methods using heuristic pattern matching motivated by weakest precondition and strongest postcondition theory. Through evaluation on 312 methods from Apache Commons Lang, we demonstrated that:

1. **High-quality specifications can be inferred automatically.** Our tool achieves 94.2% precision and 89.3% recall for preconditions, and 87.6% precision and 78.1% recall for postconditions, generating annotations for 99.0% of methods.

2. **Specifications significantly improve test generation.** LLM-based test generation guided by inferred specifications produces 272% more tests and achieves 40.7 pp higher mutation scores than signature-only baselines ($p < 0.001$, $d = 2.41$).

3. **Effectiveness varies by category.** Control Flow and Utility methods benefit most from specification guidance, while simpler accessors show more modest gains.

Our results show that automated heuristic inference removes the primary barrier to specification adoption—manual effort [25, 52]—while complementing documentation-based approaches [3]. With 88ms average analysis time per file, CI/CD integration is feasible.

The tool's latest enhancements address previously identified completeness limitations: while-loop and for-each invariant inference now applies the same accumulator and variable-relationship analyses used for standard for-loops; a built-in standard library specification database enables contract propagation for common Java API methods; and branch-aware conditional postcondition inference produces disjunctive and implication-based postconditions for methods with branching return logic.

Future work includes further enhanced loop invariant inference using machine learning [48] or hybrid static-dynamic techniques [21], object-oriented extensions (inheritance, polymorphism, class invariants), multi-language support, and interactive developer-driven refinement [13]. Evaluation on larger and more diverse codebases is needed to strengthen generalizability claims. The replication package, including all source code, data, and analysis scripts, is available to support future research.

## Data Accessibility

A complete replication package is available at `https://github.com/[anonymized]/jml-inferrer`, containing the tool source code, the 312-method dataset with categorization, all inferred specifications, generated test suites for phases P1–P4, raw mutation testing logs, statistical analysis scripts, and manual validation data.

## Acknowledgments

This research was supported by the University of Queensland Cyber initiative. We thank the anonymous reviewers for their constructive feedback.

## Conflict of Interest

The authors declare no conflict of interest.

## Author Contributions

**Brendan Edmonds**: Conceptualization, Methodology, Software, Validation, Investigation, Data Curation, Writing - Original Draft, Visualization. **Mark Utting**: Conceptualization, Methodology, Writing - Review & Editing, Supervision.

## ORCID

Brendan Edmonds https://orcid.org/0000-0000-0000-0000
Mark Utting https://orcid.org/0000-0000-0000-0000

## References

[1] June Andronick, Ross Jeffery, Gerwin Klein, Rafal Kolanski, Mark Staples, He Zhang, and Liming Zhu. Large-scale formal verification in practice: A process perspective. In *34th International Conference on Software Engineering (ICSE)*, pages 1002–1011, 2012. doi: 10.1109/ICSE.2012.6227120.

[2] Mike Barnett and K. Rustan M. Leino. Weakest-precondition of unstructured programs. In *Workshop on Program Analysis for Software Tools and Engineering (PASTE)*, pages 82–87, 2005. doi: 10.1145/1108792.1108813.

[3] Arianna Blasi, Alberto Goffi, Konstantin Kuznetsov, Alessandra Gorla, Michael D. Ernst, Mauro Pezzè, and Sergio Delgado Castellanos. Translating code comments to procedure specifications. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA '18, pages 242–253. ACM, July 2018. doi: 10.1145/3213846.3213872. URL http://dx.doi.org/10.1145/3213846.3213872.

[4] Chandrasekhar Boyapati, Sarfraz Khurshid, and Darko Marinov. Korat: Automated testing based on Java predicates. In *International Symposium on Software Testing and Analysis (ISSTA)*, pages 123–133, 2002. doi: 10.1145/566172.566191.

[5] Cristian Cadar, Daniel Dunbar, and Dawson Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 209–224, 2008.

[6] Cristiano Calcagno, Dino Distefano, Jérémy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O'Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. Moving fast with software verification. In *NASA Formal Methods Symposium (NFM)*, pages 3–11, 2015. doi: 10.1007/978-3-319-17524-9_1.

[7] Patrice Chalin, Joseph R. Kiniry, Gary T. Leavens, and Erik Poll. Beyond assertions: Advanced specification and verification with JML and ESC/Java2. *Formal Methods for Components and Objects*, 4111:342–363, 2006. doi: 10.1007/11804192_16.

[8] Bei Chen et al. CodeT: Code generation with generated tests. In *International Conference on Learning Representations (ICLR)*, 2023.

[9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[10] Henry Coles. PiTest: Mutation testing system for Java. https://pitest.org, 2016. Accessed: 2025-04-14.

[11] Patrick Cousot. A gentle introduction to formal verification of computer systems by abstract interpretation. *Logics and Languages for Reliability and Security*, 25:1–29, 2010. doi: 10.3233/978-1-60750-100-8-1.

[12] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *ACM Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977. doi: 10.1145/512950.512973.

[13] Patrick Cousot, Radhia Cousot, and Francesco Logozzo. Precondition inference from intermittent assertions and application to contracts on collections. In *International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI)*, pages 150–168, 2011. doi: 10.1007/978-3-642-18275-4_12.

[14] Saulo S. de Toledo, Antonio Martini, and Dag I. K. Sjøberg. Improving agility by managing shared libraries in microservices. In *Agile Processes in Software Engineering and Extreme Programming – Workshops*, pages 195–202. Springer International Publishing, 2020. doi: 10.1007/978-3-030-58858-8_21.

[15] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 423–435, 2023. doi: 10.1145/3597926.3598067.

[16] Edsger W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975. doi: 10.1145/360933.360975.

[17] Edsger W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1976.

[18] Natalia Dragan, Michael L. Collard, and Jonathan I. Maletic. Reverse engineering method stereotypes. In *22nd IEEE International Conference on Software Maintenance (ICSM)*, pages 24–34, 2006. doi: 10.1109/ICSM.2006.68.

[19] Vijay D'Silva, Daniel Kroening, and Georg Weissenbacher. A survey of automated techniques for formal software verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 27(7):1165–1178, 2008. doi: 10.1109/TCAD.2008.923410.

[20] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. Can large language models transform natural language intent into formal method postconditions? In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1–12, 2024. doi: 10.1145/3660773.

[21] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1–3):35–45, 2007. doi: 10.1016/j.scico.2007.01.015.

[22] Cormac Flanagan, K. Rustan M. Leino, Mark Lillibridge, Greg Nelson, James B. Saxe, and Raymie Stata. Extended static checking for Java. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 234–245, 2002. doi: 10.1145/512529.512558.

[23] Gordon Fraser and Andrea Arcuri. EvoSuite: Automatic test suite generation for object-oriented software. In *19th ACM SIGSOFT Symposium and 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, pages 416–419, 2011. doi: 10.1145/2025113.2025179.

[24] Alberto Goffi, Alessandra Gorla, Michael D. Ernst, and Mauro Pezzè. Automatic generation of oracles for exceptional behaviors. In *International Symposium on Software Testing and Analysis (ISSTA)*, pages 213–224, 2016. doi: 10.1145/2931037.2931061.

[25] Anthony Hall. Seven myths of formal methods. *IEEE Software*, 7(5):11–19, 1990. doi: 10.1109/52.57887.

[26] Osman Hasan and Sofiène Tahar. Formal verification methods. In *Encyclopedia of Information Science and Technology, Third Edition*, pages 7162–7170. IGI Global, 2015. doi: 10.4018/978-1-4666-5888-2.ch705.

[27] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969. doi: 10.1145/363235.363259.

[28] David Hovemeyer and William Pugh. Finding bugs is easy. In *ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 92–106, 2004. doi: 10.1145/1028664.1028717.

[29] JavaParser Team. JavaParser: Java parser and abstract syntax tree. https://javaparser.org, 2024. Accessed: 2025-01-15.

[30] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515*, 2024.

[31] James C. King. Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394, 1976. doi: 10.1145/360248.360252.

[32] Gerwin Klein et al. Comprehensive formal verification of an OS microkernel. *ACM Transactions on Computer Systems*, 32(1):2:1–2:70, 2014. doi: 10.1145/2560537.

[33] Gary T. Leavens, Albert L. Baker, and Clyde Ruby. Preliminary design of JML: A behavioral interface specification language for Java. *ACM SIGSOFT Software Engineering Notes*, 31(3):1–38, 2006. doi: 10.1145/1127878.1127884.

[34] Gary T. Leavens et al. JML reference manual. http://www.jmlspecs.org, 2013. Accessed: 2025-01-15.

[35] Caroline Lemieux et al. CodaMOSA: Escaping coverage plateaus in test generation with pre-trained large language models. In *45th International Conference on Software Engineering (ICSE)*, pages 919–931, 2023. doi: 10.1109/ICSE48619.2023.00085.

[36] Wei Ma et al. SpecGen: Automated generation of formal program specifications via large language models. *arXiv preprint arXiv:2401.08807*, 2024.

[37] Daniel Matichuk, Toby Murray, June Andronick, Ross Jeffery, Gerwin Klein, and Mark Staples. Empirical study towards a leading indicator for cost of formal software verification. In *37th IEEE International Conference on Software Engineering (ICSE)*, pages 722–732, 2015. doi: 10.1109/ICSE.2015.76.

[38] Bertrand Meyer. Applying "design by contract". *Computer*, 25(10):40–51, 1992. doi: 10.1109/2.161279.

[39] Carroll Morgan. *Programming from Specifications*. Prentice-Hall, USA, 1990. ISBN 978-0-13-726225-0.

[40] Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball. Feedback-directed random test generation. In *29th International Conference on Software Engineering (ICSE)*, pages 75–84, 2007. doi: 10.1109/ICSE.2007.37.

[41] Rahul Pandita, Xusheng Xiao, Hao Zhong, Tao Xie, Stephen Oney, and Amit Paradkar. Inferring method specifications from natural language API descriptions. In *34th International Conference on Software Engineering (ICSE)*, pages 815–825, 2012. doi: 10.1109/ICSE.2012.6227137.

[42] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. *Advances in Computers*, 112:275–378, 2019. doi: 10.1016/bs.adcom.2018.03.015.

[43] Dennis K. Peters and David Lorge Parnas. Using test oracles generated from program documentation. *IEEE Transactions on Software Engineering*, 24(3):161–173, 1998. doi: 10.1109/32.667877.

[44] Max Schäfer et al. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50(1):85–105, 2024. doi: 10.1109/TSE.2023.3334955.

[45] Jonathan Sillito, Gail C. Murphy, and Kris De Volder. Asking and answering questions during a programming change task. *IEEE Transactions on Software Engineering*, 34(4):434–451, 2008. doi: 10.1109/TSE.2008.26.

[46] Frank Tip, Peter F. Sweeney, Chris Laffra, Aldo Eisma, and David Streeter. Practical extraction techniques for Java. *ACM Transactions on Programming Languages and Systems*, 24(6):625–666, 2002. doi: 10.1145/586088.586090.

[47] Xinyu Wang, Isil Dillig, and Ruzica Piskac. PreInfer: Inferring preconditions via symbolic execution. In *ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 779–793, 2017. doi: 10.1145/3133876.

[48] Yi Wei, Carlo A. Furia, Nikolay Kazmin, and Bertrand Meyer. Inferring better contracts. In *33rd International Conference on Software Engineering (ICSE)*, pages 191–200, 2011. doi: 10.1145/1985793.1985820.

[49] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012. doi: 10.1007/978-3-642-29044-2.

[50] Jim Woodcock, Peter Gorm Larsen, Juan Bicarregui, and John Fitzgerald. Formal methods: Practice and experience. *ACM Computing Surveys*, 41(4):19:1–19:36, 2009. doi: 10.1145/1592434.1592436.

[51] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. No more manual tests? evaluating and improving ChatGPT for unit test generation. *arXiv preprint arXiv:2305.04207*, 2024.

[52] Daniel Zimmermann and Heike Wehrheim. Increasing the practicality of formal methods for devops with lightweight specifications. In *Formal Methods for Industrial Critical Systems (FMICS)*, volume 11815 of *LNCS*, pages 3–20. Springer, 2019. doi: 10.1007/978-3-030-30942-8_1.

**Algorithm 1** Specification Inference via Pattern Matching

**Require:** Method AST node $M$
**Ensure:** Preconditions $Pre$, Postconditions $Post$, LoopInvariants $LI$
$Pre \leftarrow \emptyset$; $Post \leftarrow \emptyset$; $LI \leftarrow \emptyset$
{Infer preconditions from early guards}
**for** each statement $S$ in $M$.body **do**
  **if** $S$ is null check `if (p == null) throw/return` **then**
    $Pre \leftarrow Pre \cup$ {`p != null`}
  **else if** $S$ is range check `if (p < 0 || p >= n) throw` **then**
    $Pre \leftarrow Pre \cup$ {`p >= 0`, `p < n`}
  **end if**
**end for**
{Infer postconditions from return statements}
**for** each `return` $e$ in $M$.body **do**
  **if** $e$ is field access `this.f` **then**
    $Post \leftarrow Post \cup$ {`\result == this.f`}
  **else if** $e$ is new object `new T(...)` **then**
    $Post \leftarrow Post \cup$ {`\result != null`}
  **else if** $e$ involves `\old` pattern **then**
    $Post \leftarrow Post \cup$ {derive relationship}
  **end if**
**end for**
{Infer conditional postconditions from branching returns}
**for** each `if-else` or ternary in $M$.body with returns in both branches **do**
  **if** both branches return literals $a$, $b$ **then**
    $Post \leftarrow Post \cup$ {`\result == a || \result == b`}
  **else if** condition is null check on parameter $p$ **then**
    $Post \leftarrow Post \cup$ {$p$ `!= null ==> \result != null`}
  **end if**
**end for**
{Infer loop invariants}
**for** each loop $L$ in $M$.body **do**
  $LI \leftarrow LI \cup$ InferLoopInvariant($L$)
**end for**
**return** $(Pre, Post, LI)$